

第 6 章

检查代码

软件测试不仅仅限于像第4章“检查产品说明书”和第5章“带上眼罩测试软件”那样把产品说明书和程序当做黑盒子来对待。如果具有编程经验，即使只有一点，就可以对软件的体系结构和代码进行测试。

在某些行业中，此类验证不如黑盒测试通用。然而，如果测试军队、金融、工业自动化、医药类软件，或者有幸在组织严格的开发模式下工作，在代码的级别验证产品就是例行公事。如果在测试软件的安全问题，那么这是必须进行的。

本章讲述进行设计和代码验证的基础。对于测试新手，这可能不是首要任务，但是如果对编程感兴趣，这就是必由之路。

本章重点包括：

- 静态白盒测试的好处
- 各种类型的静态白盒测试综述
- 编码规范和标准
- 如何从整体审查代码错误

6.1 静态白盒测试：检查设计和代码

还记得第4章所讲的静态测试和白盒测试的定义吗？静态测试是指测试非运行部分——检验和审查。白盒（或者称为透明盒）测试是指访问代码，能够查看和审查。

静态白盒测试是在不执行软件的条件下有条理地仔细审查软件设计、体系结构和代码，从而找出软件缺陷的过程，有时称为结构化分析。

进行静态白盒测试的首要原因是尽早发现软件缺陷，以找出动态黑盒测试难以发现或隔离的软件缺陷。在开发过程初期让测试小组集中精力进行软件设计的审查非常有价值。

进行静态白盒测试的另外一个好处是，为黑盒测试员在接受软件进行测试时设计和应用测试用例提供思路。他们可能不必了解代码的细节，但是通过听审查评论，可以确定有问题或者容易产生软件缺陷的特性范围。

注意 开发小组负责静态白盒测试的人员不是固定的。在某些小组中，程序员就是组织和执行审查的人员，软件测试员被邀请作为独立的观察者。还有一些小组中，软件测试员是该任务的执行人，要求编写代码的程序员和其他同事帮助审查。总之，哪一种方式都可以。这取决于开发小组选择哪一种方式更适合自身的情况。

对于静态白盒测试最不幸的是常常不能善始善终。许多小组错误地认为这样耗时太多、费用太高、没有产出。这些都不对——与产品接近完工时的有选择性的测试，找出、甚至找不出软件缺陷相比。问题在于人们一般认为程序员的任务是编写代码，而任何破坏代码编写效率的事情都会减缓开发过程。

所幸的是，形势变了。许多公司意识到早期测试的好处，并招聘和培训程序员和测试员进行白盒测试。这不是高新的学科（除非有人设计成这样），但是起步要求了解一些基本技术。假如有兴趣进一步深入，那么机会是无限的。

6.2 正式审查

正式审查（formal review）就是进行静态白盒测试的过程。正式审查的含义很广，从两个程序员之间的简单交谈，到软件设计和代码的详细、严格检查均属于此过程。

正式审查有4个基本要素：

- **确定问题。**审查的目的是找出软件的问题——不仅是出错的项目，还包括遗漏项目。全部的批评应该直指代码或设计，而不是其设计实现者。参与者之间不应该相互指责。应把自我意识、个人情绪和敏感丢在一边。
- **遵守规则。**审查要遵守一套固定的规则，规则可能设定要审查的代码量（通常有数百行），花费多少时间（数小时），哪些内容要做评价等。其重要性在于参与者了解自己的角色、目标是什么。这有助于使审查进展得更加顺利。
- **准备。**每一个参与者都为审查做准备，并尽自己的力量。根据审查的类型，参与者可能扮演不同的角色。他们需要了解自己的责任和义务，并积极参与审查。在审查过程中找出的问题大部分是在准备期间发现的，而不是实际审查期间。
- **编写报告。**审查小组必须做出审查结果的书面总结报告，并使报告便于开发小组的成员使用。审查会议结果必须尽快告诉别人——诸如发现了多少问题，在哪里发现的，等等。

进行正式审查要按照已经建立起来的过程执行。随意“聚在一起复查代码”是不够的，实际上还会造成危害。如果执行过程随意，就会遗漏软件缺陷，参与者很可能感觉这样做是在浪费时间。

如果审查正确地进行，就可以证明这是早期发现软件缺陷的好方法。它们好比是一张初始的捕虫网（见图6-1），在开始阶段捕捉大一些的虫子。诚然，小虫子仍然能够穿过，但是它们将在下一阶段被网孔编织得更细小的捕虫网抓住。

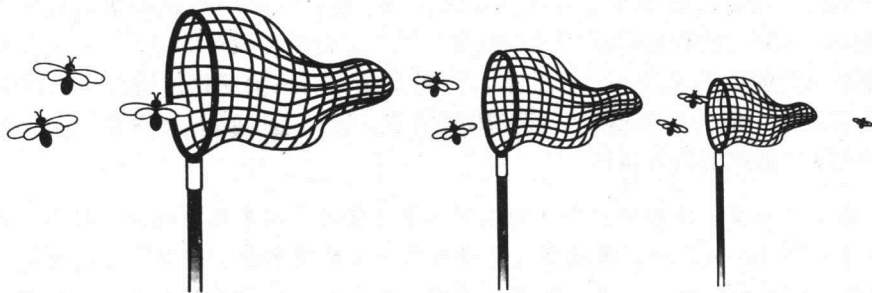


图6-1 正式审查是捕捉飞虫的第一张网

除了发现问题，坚持正式审查还有一些间接效果：

- **交流。**正式报告中未包含的信息得以交流。例如，黑盒测试员可以洞察问题所在。缺少经验的程序员可以向有经验的程序员学习新技术。管理员对于项目如何跟上进度更加心中有数。
- **质量。**程序员的代码经过逐个功能、逐行代码仔细复查，常常会使程序员变得更加仔细。这不是说他粗心大意——只是说如果他知道自己的工作要被他人仔细审查，就会多花一些心思保证正确性。
- **小组同志化。**如果审查正确进行，就会建立软件测试员和程序员对双方技艺的相互尊重，并且更好地了解相互的工作及需求。
- **解决方案。**尽管是否讨论解决方案取决于审查的规则，但是解决方案应该用于处理严重问题。在审查的范围之外讨论解决方案也许更有效。

这些间接好处虽然不可依赖，但是确实存在。在许多小组中，由于某种原因，成员之间自始至终独立工作。正式审查是把大家聚在一起讨论同一个项目问题的良机。

6.2.1 同事审查

召集小组成员进行初次正式审查最简单的方法是通过同事审查的方式。这是要求最低的正式方法，有时称为伙伴审查。这处方法大体类似于“如果你给我看你的，我也给你看我的”类型的讨论。

同事审查常常仅在编写代码或设计体系结构的程序员，以及充当审查者的其他一两个程序员和测试员之间进行。这个小团体只是在一起审查代码，寻找问题和失误。为了保证审查的高效率（不致流于休息闲聊），所有的参与者要切实保证正式审查的4个关键要素：查找问题、遵守规则、审查准备和编写报告。因为同事审查是非正式的，这些要素常常大打折扣。即便如此，聚集起来讨论代码也能找出软件缺陷。

6.2.2 走查

走查（Walkthrough）是比同事审查更正规化的下一步。走查中编写代码的程序员向5人小组或者其他程序员和测试员组成的小组做正式陈述。审查人员应该在审查之前接到软件拷贝，以便检查并编写备注和问题，在审查过程中提问。审查人员之中至少有一位资深程序员是很重要的。

陈述者逐行或者逐个功能地通读代码，解释代码为什么且如何工作。审查人员聆听叙述，提出有疑义的问题。由于公开陈述的参与人数要多于同事审查，因此，为审查做好准备和遵守规则是非常重要的。同样重要的是审查之后，表述者编写报告说明发现了哪些问题，计划如何解决发现的软件缺陷。

6.2.3 检验

检验（inspections）是最正式的审查类型，具有高度组织化，要求每一个参与者都接受训练。检验与同事审查和走查的不同之处在于表述代码的人——表述者（presenter）或者宣读者（reader）——不是原来的程序员。这就迫使他学习和了解要表述的材料，从而有可能在检验会议上提出不同的看法和解释。

其余的参与者称为检验员 (inspector)，其职责是从不同的角度，例如用户、测试员或者产品支持人员的角度审查代码。这有助于从不同视角来审查产品，通常可以指出不同的软件缺陷。检查员甚至要担负着倒过来审查代码的责任——也就是说，从尾至头——确保材料的彻底和完整。

有些检验员还同时被委任为会议协调员 (moderator) 和会议记录员 (recorder)，以保证检验过程遵守规则及审查有效进行。

召开检验会议之后，检验员可能再次碰头讨论他们发现的不足之处，并与会议协调员共同准备一份书面报告，明确解决问题所必须重做的工作。然后程序员进行修改，由会议协调员验证修改结果。根据修改的范围和规模以及软件的关键程度，可能还需要进行重新检验，以便找到其余的软件缺陷。

检验经证实是所有软件交付内容中，特别是设计文档和代码中发现软件缺陷非常有效的方法，随着公司和产品开发小组发现其好处多多而日趋流行。

6.3 编码标准和规范

在正式审查中，检验员查找代码中的问题和缺漏。这些是典型的言行不一的软件缺陷，最佳方式是通过仔细代码分析发现——资深程序员和测试员很善于此道。

还有一些问题是代码虽然可以正常运行，但是编写不符合某种标准或规范。这相当于写的东西可以被理解和表述观点，但是不符合语言的语法和文法规则。标准是建立起来、经过修补和必须遵守的规则——做什么和不做什么。规范是建议最佳做法、推荐更好的方式。标准没有例外情况，缺少结构化的放弃步骤。规范就要松一些。

可以运行，甚至在测试中也表现稳定的一些软件，因为不符合某项标准而仍然被认为有问题，真是令人感到奇怪。然而，有三个重要的原因要坚持标准或规范：

- **可靠性。**事实证明按照某种标准或规范编写的代码比不这样做的代码更加可靠和安全。
- **可读性/维护性。**符合设备标准和规范的代码易于阅读、理解和维护。
- **移植性。**代码经常需要在不同的硬件中运行，或者使用不同的编译器编译。如果代码符合设备标准，迁移到另一个平台就会轻而易举，甚至完全没有障碍。

项目要求可能从严格遵守国家或国际标准的，到松散符合小组内部规范的，不一而足。重要的是开发小组在编程过程中拥有标准和规范，并且这些标准和规范被正式审查验证。

6.3.1 编程标准和规范示例

图6-2给出了一个针对C语言的goto, while和if-else语句的编程标准示例。不正确地使用这些语句常常导致缺陷众多的代码，大多数编程标准显式设立了使用它们的规则。

标准由4个主要部分组成：

- **标题。**描述标准包含的主题。
- **标准（或者规范）。**描述标准或规范内容，解释哪些允许哪些不允许。
- **解释说明。**给出标准背后的原因，以使程序员理解这为什么这样作是好的编程习惯。
- **示例。**给出如何使用标准的简单程序示例。这不是必需的。

图6-3是一个针对C++中所用的C语言特性的规范示例。注意语言的细微差别。在本例中，它一开始就说“设法避免”。规范不如标准严格，因此在碰到类似情况时有一定的灵活性。

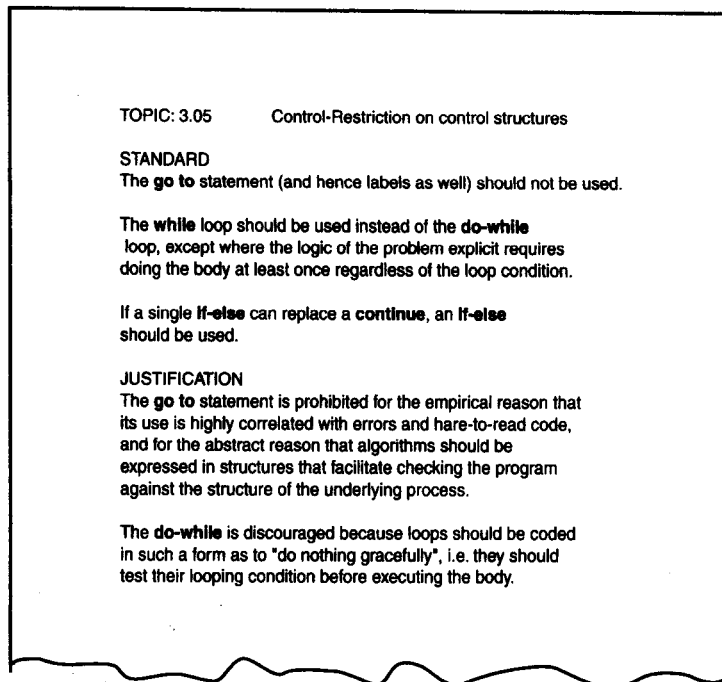


图6-2 编码标准示例解释说明了一些语言控制结构的用法（取材于Thomas Plum和Dan Saks著，Plum Hall公司1991年出版的《C++Programming Guidelines》）

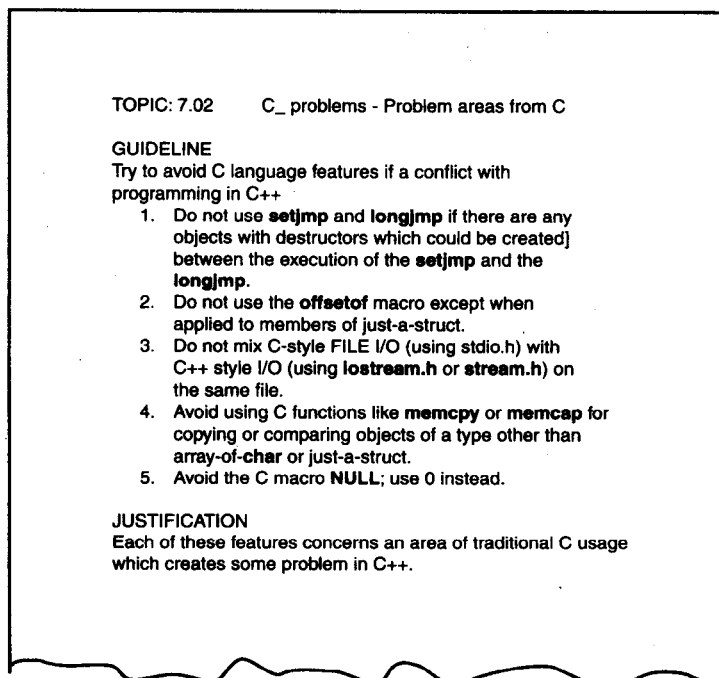


图6-3 说明在C++中如何使用某些C语言特性的编程规范示例。（取材于Thomas Plum和Dan Saks著，Plum Hall公司1991年出版的《C++Programming Guidelines》）

这是风格问题

有标准,有规范,然后就有风格。从软件质量和测试的角度看,风格不是问题。

每一个程序员,就像书的作者和艺术家一样,都有自己独特的风格。虽然规则要遵守,语言用法要一致,但是软件的编写者仍然很容易辨认。

不同之处是风格。在编程中,风格可能是注释的冗长程度和变量命名习惯,还可能是循环结构选择哪一种编排的方式。风格是代码的外表和感觉。

有些小组制定代码风格方面的标准和规范(例如代码的缩进),使代码从外表和感觉看起来都不太随意。软件测试员注意,在对软件进行正式审查时,测试和注解的对象仅限于错误和缺漏,而不管是否坚持标准或者规范。仔细想一下要报告的内容确实是问题,还仅仅是不同意见、不同风格。后者不是软件缺陷。

6.3.2 获取标准

如果项目由于自身性质必须符合一组编程标准,或者只是有兴趣检查软件代码,看它与公开发行的标准或者规范的符合程度,有一些资源可供参考。

大多数计算机语言和信息技术国家和国际标准可以通过以下站点获得:

- 美国国家标准学会 (ANSI): www.ansi.org
- 国际工程协会 (IEC): www.iec.org
- 国际标准化组织 (ISO): www.iso.ch
- 信息技术标准国家委员会 (NCITS): www.ncits.org

以下专业组织还提供演示程序规范和最佳实践的文档:

- 美国计算机协会 (ACM): www.acm.org
- 电子电气工程学会 (IEEE): www.ieee.org

还可以向销售编程工具软件的供货商索取信息。他们通常有出版的标准和规范,一般不收费,最多收少量的费用。

6.4 通用代码审查清单

本章余下部分讲述关于静态白盒测试在正式审查中验证软件应该查找的问题。这些清单^①是将代码与标准或规范比较,确保代码符合项目的设计要求。

为了真正理解和运用这些清单,需要具有一些编程经验。如果没有做过多少编程,最好先读一本入门书,例如Sams出版社出版的《Sams Teach Yourself Beginning Programming in 24 Hours》,然后才能详细审查程序代码。

6.4.1 数据引用错误

数据引用错误是指使用未经正确声明和初始化的变量、常量、数组、字符串或记录而导致的软件缺陷。

- 是否引用了未初始化的变量? 查找遗漏之处与查找错误同等重要。

① 这些清单条目取材于《Software Testing in the Real World: Improving the Process》一书第198~201页。Edward kit 1995年。

- 数组和字符串的下标是整数值吗？下标总是在数组和字符串长度范围之内吗？
- 在检索操作或者引用数组下标时是否包含“丢掉一个”这样的潜在错误？不要忘了第5章表5-1中的代码。
- 是否在应该使用常量的地方使用了变量——例如在检查数组边界时？
- 变量是否被赋予不同类型的值？例如，无意中使代码为整形变量赋予一个浮点数值？
- 为引用的指针分配内存了吗？
- 一个数据结构是否在多个函数或者子程序中引用，在每一个引用中明确定义结构了吗？

注意 数据引用错误是缓冲区溢出的主要原因——一个造成许多软件安全问题的缺陷。

第13章“软件安全性测试”将更详细地讨论该问题。

6.4.2 数据声明错误

数据声明缺陷产生的原因是不正确地声明或使用变量和常量。

- 所有变量都赋予正确的长度、类型和存储类了吗？例如，本应声明为字符串的变量声明为字符数组了吗？
- 变量是否在声明的同时进行了初始化？是否正确初始化并与其类型一致？
- 变量有相似的名称吗？这基本上不算软件缺陷，但有可能是程序中其他地方出现名称混淆的信息。
- 存在声明过、但从未引用或者只引用过一次的变量吗？
- 所有变量在特定模块中都显式声明了吗？如果没有，是否可以理解为该变量与更高级别的模块共享？

6.4.3 计算错误

计算或者运算错误实质上是糟糕的数学问题。计算无法得到预期结果。

- 计算中是否使用了不同数据类型的变量，例如将整数与浮点数相加？
- 计算中是否使用了类型相同但长度不同的变量——例如，将字节与字相加？
- 计算时是否了解和考虑到编译器对类型或长度不一致的变量的转换规则？
- 赋值的目的变量是否小于赋值表达式的值？
- 在数值计算过程中是否可能出现溢出？
- 除数/模是否可能为零？
- 对于整型算术运算，处理某些计算（特别是除法）的代码是否会导致精度丢失？
- 变量的值是否超过有意义的范围？例如，可能性的计算结果是否小于0%或者大于100%？
- 对于包含多个操作数的表达式，求值的次序是否混乱，运算优先级对吗？需要加括号使其清晰吗？

6.4.4 比较错误

小于、大于、等于、不等于、真、假。比较和判断错误很可能是由于边界条件问题。

- 比较得正确吗？虽然听起来简单，但是比较应该是小于还是小于或等于常常发生混淆。

- 存在分数或者浮点值之间的比较吗？如果有，精度问题会影响比较吗？1.00000001和1.00000002极其接近，它们相等吗？
- 每一个逻辑表达式都正确表达了吗？逻辑计算按预计的进行了吗？求值次序有疑问吗？
- 逻辑表达式的操作数是逻辑值吗？例如，是否包含整数值的整型变量用于逻辑计算中？

6.4.5 控制流程错误

控制流程错误的原因是编程语言中循环等控制结构未按预期方式工作。它们通常由计算或者比较错误直接或间接造成。

- 如果程序包含begin...end和do...while等语句组，end是否明确给出并与语句组对应？
- 程序、模块、子程序和循环能否终止？如果不能，可以接受吗？
- 可能存在永远不停的循环吗？
- 循环是否可能永不执行？如果是这样，可以接受吗？
- 如果程序包含像switch...case语句这样的多个分支，索引变量能超出可能的分支数目吗？如果超出，该情况能正确处理吗？
- 是否存在“丢掉一个”错误，导致循环意外的流程？

6.4.6 子程序参数错误

子程序参数错误的来源是软件子程序不正确地传递数据。

- 子程序接收的参数类型和大小与调用代码发送的匹配吗？次序正确吗？
- 如果子程序有多个入口点，引用的参数是否与当前入口点没有关联？
- 常量是否当做形参传递，在子程序中被意外改动？
- 子程序更改了仅作为输入值的参数吗？
- 每一个参数的单位是否与相应的形参匹配——例如，英尺对米？
- 如果存在全局变量，在所有引用子程序中是否有相同的定义和属性？

6.4.7 输入/输出错误

输入/输出错误包括文件读取、接受键盘或者鼠标输入以及向打印机或者屏幕等输出设备写入错误。下列条目非常简单、通用，应该在使用时补充，以涵盖所测试的软件。

- 软件是否严格遵守外部设备读写数据的专用格式？
- 文件或者外设不存在或者未准备好的错误情况有处理吗？
- 软件是否处理外部设备未连接、不可用，或者读写过程中存储空间占满等情况？
- 软件以预期方式处理预计的错误吗？
- 检查错误提示信息的准确性、正确性、语法和拼写了吗？

6.4.8 其他检查

这个压轴清单定义了一些不适合放在其他类别的条目。这不是为了完整，而是提示为定制软件项目清单应该加入的内容。

- 软件是否使用其他外语？是否处理扩展ASCII字符？是否需要用统一编码取代ASCII？

- 软件是否要移植到其他编译器和CPU，具有这样做的许可吗？如果没有计划或者测试，那么，移植性可能成为一个大难题。
- 是否考虑了兼容性，以使软件能够运行于不同数量的可用内存、不同的内部硬件（例如图形和声卡）、不同的外设（例如打印机和调制解调器）？
- 程序编译是否产生“警告”或者“提示”信息？这些信息通常指示进行了有疑问的处理。纯粹主义者可能认为警告信息是不可接受的。

6.5 小结

检查代码——静态白盒测试——被证实是早期发现软件缺陷最有效的方法。虽然这是一项需要大量准备工作才能有成效的任务，但是许多研究表明花费的时间与得到的好处相比是值得的。为了使这项任务更有吸引力，现在有了能自动执行大量静态白盒测试工作的商业软件，即静态分析程序。该程序读入程序的源文件，并根据公开标准和自定义规范进行检查。编译器也提高了能力，如果启用所有等级的错误检查，它将捕捉到前面通用代码审查清单列出的许多问题，有些编译器甚至不允许使用具有安全问题的函数。这些工具不是取消代码审查或者检查任务——只是使任务更容易完成，并给软件测试员更多时间来挖掘更深的软件缺陷。

如果开发小组没有在这一级进行测试，而你具有一些编程经验的话，就可以把测试想像为调查过程。程序员和经理开始可能有顾虑，不知道好处有多大——这很难说出来，比如在检验期间发现一个软件缺陷比一个月之后在黑盒测试期间发现它可以为项目节省5天时间。然而，静态白盒测试正在受到重视，而在某些开发过程中，没有它，项目就无法得到可靠的软件。

6.6 小测验

以下是帮助读者加深理解的小测验。答案参见附录A——但是不要偷看！

1. 说出进行静态白盒测试的几个好处。
2. 判断是非：静态白盒测试可以找出遗漏之处和问题。
3. 正式审查由哪些关键要素组成？
4. 除了更正式之外，检验与其他审查类型有什么重大差别？
5. 如果要求程序员在命名变量时只能使用8个字符并且首字母必须采用大写的形式，那么这是标准还是规范呢？
6. 你会采用本章的代码审查清单作为项目小组验证代码的标准吗？
7. 缓冲区溢出错误作为一个常见的安全问题属于哪一级错误？是由什么原因引起的？